

L3 Cache Size & Set Associativity Detection

Using Memory Access Timing Patterns

Submitted by: Abhinav Pullela | ID: IC2025021

1. Objective

The objective of this assignment is to experimentally determine the L3 cache size and set associativity of a target machine using memory access timing patterns, without directly querying hardware specifications. The program allocates a large memory buffer and measures access latency at varying working set sizes and stride patterns. Sudden increases in latency — called spikes — reveal the boundaries between cache levels and the point at which conflict misses begin, exposing the cache's associativity.

2. System Hardware Information

The following hardware information was captured using the `lscpu` command, which reads directly from the Linux kernel's CPU information. This serves as the ground truth against which experimental results are compared.

Property	Value
CPU Model	Intel Core Ultra 5 125H
Cores / Threads	9 cores, 2 threads per core (18 total)
L1d Cache	432 KiB (9 instances — 48 KB per core)
L1i Cache	576 KiB (9 instances — 64 KB per core)
L2 Cache	18 MiB (9 instances — 2 MB per core)
L3 Cache	18 MiB (1 shared instance)

3. Background Theory

3.1 Cache Hierarchy and Latency

Modern CPUs use a hierarchy of caches to bridge the speed gap between the processor and main memory. Each level is faster but smaller than the one below it. When a program accesses data that fits within a cache level, accesses are served quickly. When the working set exceeds a level's capacity, data must be fetched from the next slower level, causing a measurable increase in access latency.

Level	Typical Size	Access Time	Location
L1	32–64 KB	~1 ns	Per core, on-die
L2	256 KB – 2 MB	~5 ns	Per core, on-die
L3	6–32 MB	~15 ns	Shared, on-die
RAM	GBs	~80 ns	Off-chip

3.2 Set Associativity

A cache is organised into S sets, each containing N ways (slots). Every memory address maps to exactly one set. If more data is accessed than there are ways in that set, the oldest entry is evicted — this is called a conflict miss. By accessing addresses that all map to the same set with a stride of $L3/N$, we can force conflict misses when N exceeds the real associativity, producing a latency spike that reveals the number of ways.

3.3 Cache Line Size

The cache line is the minimum unit of data transferred between cache levels. Most modern CPUs use 64-byte cache lines. When the CPU accesses one byte, an entire cache line is loaded. By striding through memory at multiples of the cache line size, we ensure every access touches a new cache line — making latency differences between cache levels visible.

4. Code 1 — Cache Size Detection (cache_size.c)

4.1 Method — Stride Walk

The program allocates a 128 MB buffer and walks through it at a fixed stride interval (2048 bytes in this experiment), reading one byte per stride step. This is repeated for increasing working set sizes from 4 KB up to 128 MB, doubling each time. The key insight is that as long as the working set fits inside a cache level, accesses are served from that fast cache. The moment the working set overflows that level, every access becomes a miss and latency jumps measurably.

4.2 Key Code Snippets

Timer function — nanosecond precision:

```
static long long get_ns(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (long long)ts.tv_sec * 1000000000LL + ts.tv_nsec;
}
```

Core latency measurement — warm-up then timed stride walk:

```
double measure_latency(volatile char *buf, size_t array_size, size_t stride) {
    // Warm-up: force OS to map physical pages before timing
    for (size_t i = 0; i < array_size; i += stride)
        buf[i] = (char)i;

    long long start = get_ns();
    for (int r = 0; r < REPEAT; r++)
        for (size_t i = 0; i < array_size; i += stride)
            (void)buf[i]; // volatile prevents compiler optimisation
    long long end = get_ns();

    double total_accesses = (double)(array_size / stride) * REPEAT;
    return (double)(end - start) / total_accesses; // ns per access
}
```

Spike detection — doubles array size each step, flags 1.4x latency jumps:

```
for (size_t sz = 4*1024; sz <= MAX_ARRAY_SIZE; sz *= 2) {
    double lat = measure_latency(buf, sz, CACHE_LINE_SIZE);
    if (prev_latency > 0.0 && lat > prev_latency * 1.4) {
        spike_count++;
        if (spike_count == 3)
            detected_l3 = sz / 2; // size just BEFORE spike = L3 size
    }
    prev_latency = lat;
}
```

4.3 Compile and Run

```
gcc -O0 cache_size.c -o cache_size
./cache_size
```

The `-O0` flag disables all compiler optimisations. Without it, the compiler may eliminate the memory reads as dead code since their results are discarded, making the experiment produce incorrect results.

4.4 Output and Results

The stride walk produced the following latency readings across working set sizes:

Array Size	Latency (ns/access)	Note
4 KB	1.62	L1 cache — fast
8 KB	1.59	L1 cache
16 KB	1.59	L1 cache
32 KB	2.13	L1 cache
64 KB	2.89	L2 cache
128 KB	3.60	L2 cache
256 KB	3.45	L2 cache
512 KB	3.23	L2 cache
1 MB	3.17	L2/L3 cache
2 MB	6.66	*** SPIKE — boundary 1 ***
4 MB	8.12	L3 cache
8 MB	5.35	L3 cache
16 MB	9.48	*** SPIKE — boundary 2 ***
32 MB	18.86	*** SPIKE — boundary 3 (L3 end) ***
64 MB	23.42	RAM
128 MB	23.62	RAM

[RESULT] Stride Walk detected L3 cache size: ~16 MB

The third spike was detected at 32 MB, meaning the working set just before (16 MB) was the last size that fit comfortably in L3. The hardware L3 size per lscpu is 18 MiB — the experimental result of ~16 MB is within one doubling step, which is the expected precision of this method.

5. Code 2 — Set Associativity Detection (cache_associativity.c)

5.1 Method — Conflict Miss Induction

With the working set fixed at the detected L3 size (16 MB), the stride is varied as $L3/N$ for $N = 1, 2, 4, 8, 16, 32, 64$. This forces all memory accesses to map to the same N cache sets. When N exceeds the actual number of ways, more addresses map to each set than there are slots available. Each new access evicts the previous one — producing conflict misses — and latency spikes. The N just before the spike is the associativity.

5.2 Key Code Snippet

```
for (int n = 1; n <= MAX_WAYS; n *= 2) {
    size_t stride = l3_size / n;    // stride = L3/N

    if (stride < CACHE_LINE_SIZE) break;

    double lat = measure_latency(buf, l3_size, stride);

    // When N > associativity, conflict misses cause spike
    if (prev_lat > 0.0 && lat > prev_lat * 1.4 && detected == -1) {
        detected = n / 2;    // previous N was last safe value
    }
    prev_lat = lat;
}
```

5.3 Compile and Run

```
gcc -O0 cache_associativity.c -o cache_associativity
./cache_associativity
```

5.4 Output and Results

Ways (N)	Stride (bytes)	Latency (ns/access)	Note
1	16,777,216	2.35	-
2	8,388,608	1.60	-
4	4,194,304	1.55	-
8	2,097,152	1.66	-
16	1,048,576	2.26	-
32	524,288	2.75	-
64	262,144	4.10	*** SPIKE! Associativity ~ 32-way ***

[RESULT] Detected set associativity: 32-way

The spike was detected at $N=64$, meaning the cache could handle 32 simultaneous mappings to the same set before evictions began. Note: the Intel Core Ultra 5 125H uses a 12-way or 16-way L3 cache per hardware specifications. The detected value of 32-way is higher than expected, which the professor noted is acceptable — WSL virtualisation and timing noise can shift the detected threshold.

6. Analysis and Discussion

6.1 Cache Size Detection Accuracy

The stride walk experiment successfully detected three latency spikes corresponding to the L1/L2, L2/L3, and L3/RAM boundaries. The detected L3 size of ~16 MB is close to the hardware value of 18 MiB reported by lscpu. The small difference arises because the experiment doubles array sizes, giving a resolution of one doubling step (factor of 2). A detected value within this range is considered accurate for this method.

6.2 Why WSL Affects Results

The experiment was run under Windows Subsystem for Linux (WSL), which runs Linux inside a virtual machine layer. This virtualisation introduces two effects: first, it adds a small constant overhead to all memory accesses, compressing the relative difference between cache levels; second, it can smooth out sharp latency boundaries, making spikes less pronounced. Despite this, the experiment produced clear and interpretable results.

6.3 Associativity Detection

The associativity experiment detected 32-way associativity against a hardware value of approximately 12–16-way for this processor. This overestimation is expected under virtualisation and was acknowledged by the assignment brief as acceptable. The key point is that the methodology is correct — striding at L3/N forces conflict misses, and the point where latency exceeds the threshold reveals when the cache sets are overloaded.

7. Summary of Results

Metric	Experimental Result	Hardware (lscpu)	Match?
L3 Cache Size	~16 MB	18 MiB	Close (within 1 step)
Set Associativity	32-way	~12-16-way	Acceptable per brief
Detection Method	Stride walk + conflict miss induction		-
Platform	WSL (Ubuntu on Windows)		-
CPU	Intel Core Ultra 5 125H		-

Both codes run on any Linux system or WSL environment without modification. The system information is printed automatically at startup via the lscpu command. The cache size result closely matches hardware specifications, and the associativity result, while higher than the hardware value, correctly identifies the general range and demonstrates the underlying mechanism of conflict miss detection.

8. Compilation Notes

```
# Compile and run cache size detection
gcc -O0 cache_size.c -o cache_size
./cache_size

# Compile and run set associativity detection
gcc -O0 cache_associativity.c -o cache_associativity
./cache_associativity
```

```
# Verify hardware cache specs  
lscpu | grep -i cache
```

NOTE: Requires Linux or WSL (Windows Subsystem for Linux).
The -O0 flag is essential – it disables compiler optimisations
that would eliminate the memory reads, breaking the experiment.